

Mid-Evaluation Assessment Report

Design and Verification of a
High-Speed DDR Display Controller

Team ID: Elec-13

Team Members:

Harshit Gautam (250002031) Ayusman Dash (250002017)
Anshu Yadav (250002014) Yuvraj Diwakar (250002087)

Project Mentor:

Rucha Gadgil



Science & Technology Council, IIT Indore
Students' Gymkhana

30 June, 2026



Contents

1	Introduction & Objectives	1
1.1	What DDR is and its implementation:	1
1.2	Primary Objectives of the Project:	1
1.2.1	Technical & Functional Objectives:	1
1.2.2	Verification & Deliverable Objectives	2
2	Mathematical Timing Parameters	2
2.1	General Definitions:	2
2.2	Resolution Specific Timing Parameters	3
3	System Architecture Planning	3
3.1	SDR Domain Control Logic	3
3.2	Physical DDR Interface Integration Boundaries	4
4	RTL Implementation Status	4
4.1	Display Timing Generator	4
4.2	Coordinate Tracking State Machine	6
5	Simulation Findings & Verification Proof	7
5.1	Timing Waveform Verification	7
5.2	Boundary Condition & Bounce Verification	8
6	Conclusion & Future Work	10

1 Introduction & Objectives

1.1 What DDR is and its implementation:

Driving high-resolution external display interfaces requires an exceptional amount of data bandwidth. Relying on traditional Single Data Rate (SDR) techniques often demands pushing internal FPGA clock frequencies to unstable, high thresholds that complicate routing and violate timing closure. Furthermore, utilizing standard behavioral RTL constructs for high-speed external physical output buffers can introduce severe setup and hold violations, compromising the stability of video output streams.

To address these hardware limitations, this project focuses on the architecture and implementation of a custom High-Speed DDR Display Controller. The core system is designed to isolate high-speed parallel-to-serial data transitions from internal operations. This is accomplished by managing pattern generation and control logic in a safer, lower-frequency standard clock domain (f). The data is then serialized for physical transmission at double the data rate ($2f$) using vendor-specific DDR hardware primitives.

The current progression simply focuses on building the foundation of the project, which includes building a clear understanding of rendering the pixels on the screen taking into account certain resolutions (*e.g.* 640×480 , 1920×1080 , *etc.*), detailed timing calculations, and a fully synthesizable base Register Transfer Level (RTL) design operating solely within the standard Single Data Rate (SDR) clock domain.

1.2 Primary Objectives of the Project:

The primary objectives for the mid-evaluation are to establish a robust, verified SDR foundational layer and a comprehensive architectural roadmap for eventual DDR integration. The specific objectives are detailed below:

1.2.1 Technical & Functional Objectives:

- **Design a Synthesizable Display Timing Generator:** Implement precise counters and logic in synthesizable RTL to accurately generate Horizontal Sync (HSYNC), Vertical Sync (VSYNC), and the essential Horizontal/Vertical Blanking and "Active Video" enable signals based on chosen resolution benchmarks.
- **Implement a Coordinate Tracking State Machine:** Develop a dedicated state machine to track and continuously update the X/Y position of a dynamic 50×50 pixel block pattern.
- **Model Dynamic Edge-Collision Logic:** Embed boundary-detection mathematical logic within the coordinate state machine to capture screen boundary collisions and execute a flawless reversing "bounce" effect.
- **Perform Complete Pre-Serialization Architecture Mapping:** Graphically model and document the overall system architecture, conceptualizing the areas to be updated upon serialization from SDR domain to DDR domain.

1.2.2 Verification & Deliverable Objectives

- **Execute Testbench-Driven Simulation:** Write a comprehensive behavioral testbench to simulate the integrated core modules (Timing Generator and Coordinate Tracker) to mathematically validate logic accuracy.
- **Validate Precise Timing Metrics:** Extract simulation waveforms proving that HSYNC, VSYNC, and Active Video signals match the calculated mathematical display parameters (Pixel Clock, Front/Back Porches, and Sync pulse widths).
- **Boundary and Coordinate Behavior:** Provide an absolute simulation proof verifying that the X/Y counters correctly increment, decrement, and cleanly reverse the tracking direction upon hitting artificial screen boundaries.
- **Compile Synthesizable Codebase:** Assemble a fully operational codebase (compatible with Vivado project formats) while making sure that the code remains synthesizable.

2 Mathematical Timing Parameters

2.1 General Definitions:

In order to properly align all the pixels on the screen without any distortion or blurring, a thorough understanding of the mathematical timing parameters is required.

Primarily, a complete display frame consists of the visible region (Active Video) surrounded by a non-visible buffer area called the Blanking Interval (made up of the Front Porch, Sync Pulse, and Back Porch). Henceforth, pixel generation under any resolution via a VGA cable works on the following parameters:

- **Active Width** (H_{ACTIVE}): The total number of visible horizontal pixels per line.
- **Active Height** (V_{ACTIVE}): The total number of visible vertical lines per frame.
- **Front Porch:** Padding delay immediately after active video and just before the sync pulse.
- **Sync Pulse:** The active synchronization signal (H_{SYNC}/V_{SYNC}) that triggers the reset of the electron gun or monitor processing.
- **Back Porch:** Padding delay immediately after the sync pulse, returning the beam before the next active line/frame begins.
- **Total Dimensions** (H_{TOTAL}/V_{TOTAL}): The comprehensive boundaries of the video timeline, calculated using the sum of active and blanking regions:

$$H_{TOTAL} = H_{ACTIVE} + H_{FRONT_PORCH} + H_{SYNC_WIDTH} + H_{BACK_PORCH}$$

$$V_{TOTAL} = V_{ACTIVE} + V_{FRONT_PORCH} + V_{SYNC_WIDTH} + V_{BACK_PORCH}$$

- **Refresh Rate (RR):** The number of complete frames rendered per second (standardly targetted at 60 Hz).
- **Pixel Clock (f_{pixel}):** The raw operational frequency needed to feed individual pixel elements sequentially:

$$f_{pixel} = H_{TOTAL} \times V_{TOTAL} \times RR$$

2.2 Resolution Specific Timing Parameters

Since modern monitors usually drive displays at a resolution 1920×1080 @ 60 Hz frame refresh rate, the entire project has been designed to facilitate the same in the form of "parameters" rather than fixed values in the design source files. Below are the industry-standard mathematical timing parameters (VESA standards) configured for a standard 60 Hz frame refresh rate:

Table 1: Timing Parameter Specifications for 1920×1080 Resolution @ 60 Hz ($f_{pixel} = 148.50$ MHz)

Video Parameter	Horizontal (Pixels)	Duration	Vertical (Lines)	Duration
Active Region	1920	12.929 μs	1080	16.000 ms
Front Porch	88	592.59 ns	4	59.26 μs
Sync Pulse Width	44	296.30 ns	5	74.07 μs
Back Porch	148	996.63 ns	36	533.33 μs
Total Frame	2200	14.815 μs	1125	16.667 ms

Pixel Clock Calculation:

$$f_{pixel} = 2200 \times 1125 \times 60 \text{ Hz} = \mathbf{148.50 \text{ MHz}}$$

Internal SDR Clock Frequency:

$$f_{SDR} = \frac{148.50 \text{ MHz}}{2} = \mathbf{74.25 \text{ MHz}}$$

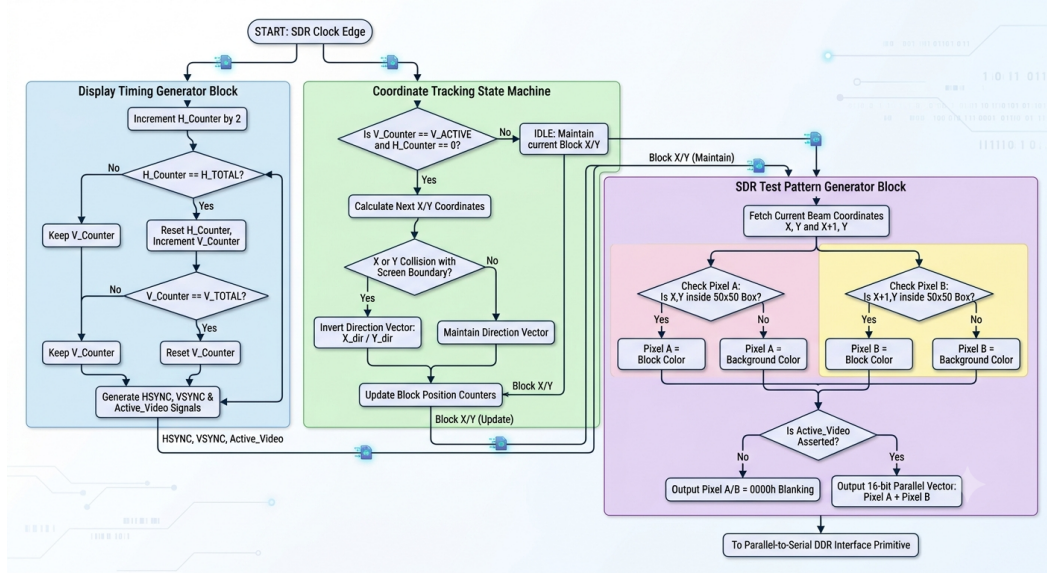
As a side note, the use of DDR is instrumental due to the fact that while a frequency of 25.175 MHz i.e. the common VGA Standard of 640×480 resolution can easily be supplied by the internal clock of an FPGA, generating a frequency of 148.50 MHz is considerably difficult just by utilizing the SDR domain and hence, DDR comes in handy for that role.

3 System Architecture Planning

3.1 SDR Domain Control Logic

According to the mid-evaluation criterion, the screen rendering logic and block generation has to be carried out in the SDR domain, which can further be updated in future to the DDR domain via the use of the ODDR primitive on the physical FPGA board. For the present scenario, the control logic handles two main processes running in parallel: the Display Frame Timing Generation (which updates every clock cycle)

and the Coordinate Tracking State Machine (which updates once per frame, typically during the vertical blanking window).



3.2 Physical DDR Interface Integration Boundaries

To transition from the current Single Data Rate (SDR) structural testing baseline to a true high-speed physical interface, a dedicated integration boundary must serialize the data. Because internal pattern logic operates on an SDR clock running at half the required pixel velocity ($f_{SDR} = f_{pixel}/2$), the pipeline calculates two distinct pixel colors concurrently on every positive edge: Pixel A and Pixel B.

The interface boundary maps these parallel variables alongside standard synchronization paths directly onto FPGA physical layer components (such as the Xilinx ODDR macro primitive). The serialization block takes the interleaved 16-bit parallel output vector $\{\text{Pixel_A_Data}, \text{Pixel_B_Data}\}$, multiplexes the content across alternating rising and falling edges of the high-speed transmitter clock, and presents a dense serial stream to the hardware pins. This isolates internal display control paths from high-frequency serialization skew and ensures timing closure during final implementation.

4 RTL Implementation Status

4.1 Display Timing Generator

The Display Timing Generator acts as the foundational clock reference for the entire synchronous pipeline. It increments nested horizontal and vertical registers sequentially to establish standard resolution protocols. Listing 1 provides the synthesizable Verilog execution for the line tracking registers and frame strobe generation.

```

1 // Horizontal Pixel Line Position Counter
2 always @(posedge clk) begin
3     if(hcount < ('htotal + 1)) begin

```

```

4         hcount <= hcount + 1;
5         enable<=1'b0;
6     end
7     else begin
8         enable <= 1'b1;
9         hcount <= 0;
10    end
11 end
12
13 // Vertical Frame Line Position Counter
14 always @(posedge clk) begin
15     if(enable == 1'b1) begin
16         if(vcount < ('vtotal + 1)) begin
17             vcount <= vcount + 1;
18             frame <= 1'b0;
19         end
20         else begin
21             vcount <= 0;
22             frame <=1'b1;
23         end
24     end
25 end

```

Listing 1: Synthesizable Sequential Row and Frame Line Counters.

The synchronization vectors (`hsynq`, `vsynq`) and the display passband area indicator (`video_active`) are continuously evaluated through concurrent combinational statements. Listing 2 outlines the control assignments and the subsequent synchronous pixel mapping logic used to project the blue target pattern.

```

1 // Sync Signals and Active Passband Extraction
2 assign hsynq = (hcount < 'hsync_val) ? 1'b1:1'b0;
3     assign vsynq = (vcount < 'vsync_val) ? 1'b1:1'b0;
4
5     wire video_active = (hcount >= 'hstart && hcount < 'hend) && (
6         vcount >= 'vstart && vcount < 'vend
7
8 // Synchronous Pattern Rendering Pipeline
9 always @(posedge clk) begin
10     if (!video_active) begin
11         obblue <= 8'd0; ored <= 8'd0; ogreen <= 8'd0;
12     end
13     else if(hcount>=boxx && hcount<=(boxx+50) && vcount>=boxy &&
14         vcount<=(boxy+50)) begin
15         obblue <= 8'd255; ogreen <= 8'd0; ored <= 8'd0;
16     end
17     else begin
18         obblue <= 8'd255; ored <= 8'd255; ogreen <= 8'd255;
19     end
20 end

```

Listing 2: Combinational Synchronization and Synchronous Pixel Rendering Pipeline.

4.2 Coordinate Tracking State Machine

The spatial orientation and boundary calculations for the rendering block are evaluated by a pre-determining coordinate-tracking finite state machine. To avoid mid-frame raster distortion and visual screen tearing, position calculation routines are executed exclusively once per complete display refresh, gated safely at the vertical boundary interval where `hcount == ('htotal - 1) && vcount == ('vtotal - 1)`. Listing 3 shows the complete synthesizable trajectory optimization logic.

```
1  always @(posedge clk) begin
2      if (hcount == ('htotal - 1) && vcount == ('vtotal - 1)) begin
3
4          // X Movement
5          if (xdir == 1'b1) begin
6              if ((boxx + 50 + 'speed) > 'hend) begin
7                  boxx <= 'hend - 50;
8              end else begin
9                  boxx <= boxx + 'speed;
10             end
11         end
12         else begin
13             if ((boxx - 'speed) < 'hstart) begin
14                 boxx <= 'hstart;
15             end else begin
16                 boxx <= boxx - 'speed;
17             end
18         end
19
20         // Y Movement
21         if (ydir == 1'b1) begin
22             if ((boxy + 50 + 'speed) > 'vend) begin
23                 boxy <= 'vend - 50;
24             end else begin
25                 boxy <= boxy + 'speed;
26             end
27         end
28         else begin
29             if ((boxy - 'speed) < 'vstart) begin
30                 boxy <= 'vstart;
31             end else begin
32                 boxy <= boxy - 'speed;
33             end
34         end
35
36         // Direction Reversal upon hitting boundary
37         if ((boxy + 50) >= 'vend)
38             ydir <= 1'b0;
39         else if (boxy <= 'vstart)
40             ydir <= 1'b1;
41
42         if ((boxx + 50) >= 'hend)
43             xdir <= 1'b0;
44         else if (boxx <= 'hstart)
45             xdir <= 1'b1;
46     end
end
```



```

47     end
48     end

```

Listing 3: Synthesizable Block Movement Logic.

5 Simulation Findings & Verification Proof

The Vivado Simulations have been compiled to account for various situations which include sync timings, active video boolean and block kinematics.

5.1 Timing Waveform Verification

The simulation waveforms were analyzed to verify the operation of the display timing generator. The results confirm the correct generation of the HSYNC, VSYNC, and Active Video signals. The following figures present the simulation waveforms and their corresponding observations:



Figure 1: HSYNC Waveform

Observation: HSYNC becomes LOW during the horizontal synchronization interval. It indicates the end of one display line and the beginning of the next.

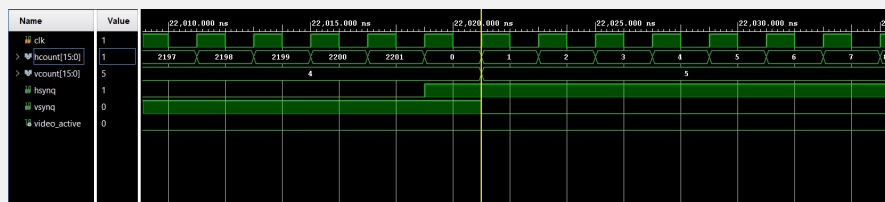


Figure 2: VSYNC Waveform

Observation: VSYNC becomes LOW during the vertical synchronization interval. It marks the completion of one display frame.

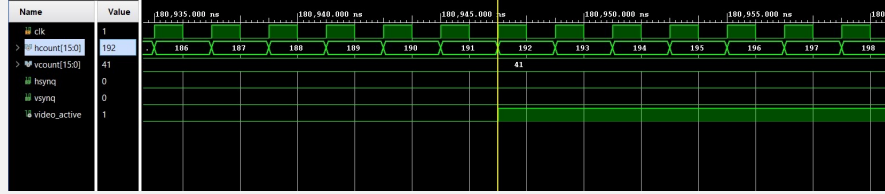


Figure 3: Active Video = 1

Observation: The Active Video signal remains HIGH within the visible display region. The RGB data are enabled only during this period.

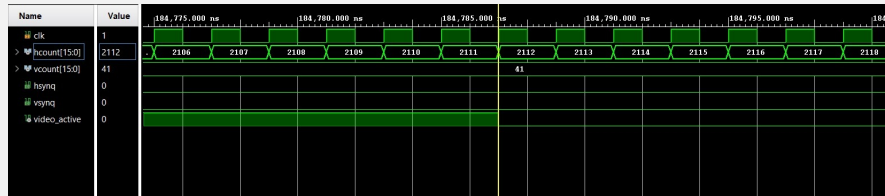


Figure 4: Active Video = 0

Observation: The Active Video signal remains LOW during the blanking intervals. RGB outputs are disabled outside the visible display area.

5.2 Boundary Condition & Bounce Verification

This section presents the simulation results for boundary detection and bounce behavior of the display controller. The figures below verify that the object correctly responds to screen edges and corners by reversing direction upon reaching display boundaries.

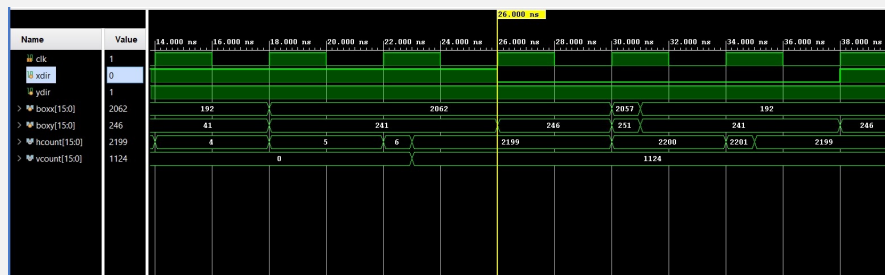


Figure 5: Right Edge

Observation: When the object reaches the right boundary of the display area, it detects the edge condition and reverses its horizontal direction, ensuring it remains within the visible screen region.

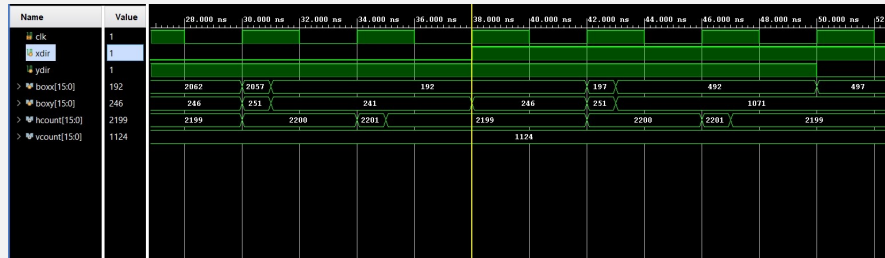


Figure 6: Left Edge

Observation: On reaching the left boundary, the object bounces back by changing its direction from left to right, maintaining proper horizontal confinement.

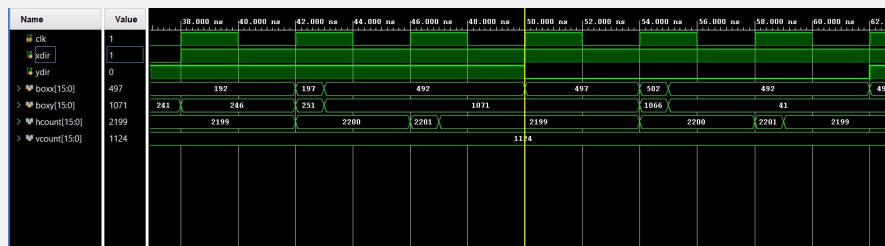


Figure 7: Top Edge

Observation: When the object reaches the top boundary, it reverses its vertical direction downward, ensuring that it remains within the display area.

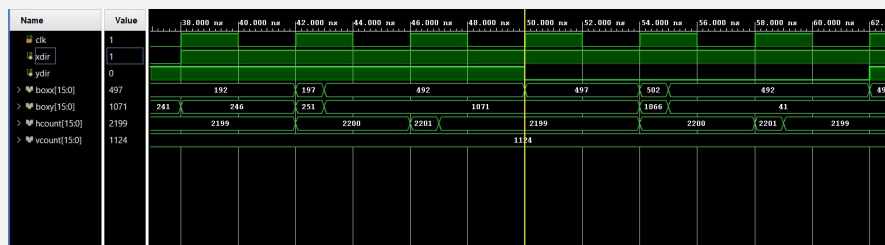


Figure 8: Bottom Edge

Observation: At the bottom boundary, the object bounces upward by reversing its vertical motion, maintaining correct screen constraints.



Figure 9: Bottom-Left Corner

Observation: At the bottom-left corner, both horizontal and vertical directions are reversed simultaneously due to dual boundary collision detection.

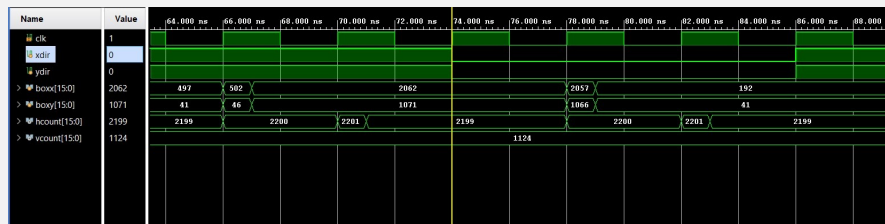


Figure 10: Bottom-Right Corner

Observation: At the bottom-right corner, the object reverses both axes' direction, ensuring correct bounce behavior at diagonal boundaries.

6 Conclusion & Future Work

The current progress encapsulates the SDR domain processing, a precise coordinate tracking and proper kinematics governing speed control and collision-reversal considerations, all the while maintaining synthesizability of code.

The next development will focus on high-speed physical layer Double Data Rate (DDR) integration. To achieve efficient 2:1 hardware serialization, a data “**Prefetching**” architecture will be implemented. Operating at half the target pixel frequency ($f/2 = 74.25$ MHz for Full HD), the pattern generator will compute two pixel streams simultaneously per clock cycle and pack them into 16-bit wide registers (e.g. [15:0] o_red1, o_red2).